Assignment 1

Question 1 (15 marks):

This question is about the term-document matrix and has two parts:

- Outline a suitable indexing structure to store the information in the matrix (note that the matrix is sparse). (10 marks)

Answer: Part A

The **Compression Sparse Row (CSR)** format is a suitable indexing structure for storing term-document matrices. CSR is highly efficient for sparse matrices, storing non-zero values and reducing storage requirements while enabling fast row access. This format is widely recognized as optimal for term-document matrices in information retrieval, as outlined in the research paper *'Comparative Analysis of Sparse Matrix Algorithms for Information Retrieval' [1].*

1. Storage Efficiency: CSR takes the least storage space on disk compared to other sparse matrix formats like Coordinate Storage (COO), Compressed Sparse Column (CSC), and Block Sparse Row (BSR). CSR stores only the non-zero elements, significantly reducing storage space.
2. Query Processing: CSR's row-oriented design efficiently retrieves documents related to specific terms which is critical for fast query response.
3. Organization: CSR stores non-zero elements in three arrays to store data efficiently, minimizing memory use while maintaining quick access to necessary information.

Key Components of CSR Format

1. **Non-Zero Values Array**: Stores the non-zero term weights (tf-idf) for each document-term pairing.
2. **Column Index Array**: Points to each non-zero term's document (column) in the matrix.
3. **Row Pointer Array**: Points to the start of each document (row) in the non-zero values and column index arrays, enabling rapid row access.

Example of CSR Format

Input: we generate a CSR matrix from the query and the documents where the rows represent the documents and the columns represent the term frequencies in the documents

	Term 1	Term 2	Term 3	Term 4	Term 5	Term 6
D1	10	20	0	0	0	0
D2	0	30	0	4	0	0
D3	0	0	50	60	70	0
D4	0	0	0	0	0	80

Step 1: Identify Non-Zero Values

- **Non-Zero Values (A)**: [10, 20, 30, 4, 50, 60, 70, 80]

Step 2: Identify Column Indices

Next, we create an array that stores the column indices corresponding to each non-zero value. The indices are based on their positions in the original matrix.

- **Column Indices (JA)**:
  - 10 (row 0, column 0) → index 0
  - 20 (row 0, column 1) → index 1
  - 30 (row 1, column 1) → index 1
  - 4 (row 1, column 3) → index 3
  - 50 (row 2, column 2) → index 2
  - 60 (row 2, column 3) → index 3
  - 70 (row 2, column 4) → index 4
  - 80 (row 3, column 5) → index 5
- **Resulting JA**: [0, 1, 1, 3, 2, 3, 4, 5]

Step 3: Create Row Pointer Array

The Row Pointer (IA) array should indicate the index position in the Values (A) array where each row's non-zero elements begin. Each entry in IA points to the start of a new row in A.

- **Row Pointer (IA)**:
  - Row 0 starts at index 0 (2 non-zero elements: 10, 20)
  - Row 1 starts at index 2 (2 non-zero elements: 30, 4)
  - Row 2 starts at index 4 (3 non-zero elements: 50, 60, 70)
  - Row 3 starts at index 7 (1 non-zero element: 80)
- **Resulting IA**: [0, 2, 4, 7, 8]

Final CSR Representation

CSR format representation of the sparse matrix:

- **Non-Zero Values (A)**: [10, 20, 30, 4, 50, 60, 70, 80]
- **Column Indices (JA)**: [0, 1, 1, 3, 2, 3, 4, 5]
- **Row Pointer (IA)**: [0, 2, 4, 7, 8]

Answer: Part B

- Outline at a high level, in pseudo-code, an algorithm to calculate the similarity of a document to a query. (5 marks)

Query Processing Algorithm

To calculate the similarity of a document to a query you can use cosine similarity. This measures the cosine angle between two vectors of the query vector and the document vector. If two vectors point in the same direction the similarity is 1 and if pointing in the opposite direction is 0.

Steps

1. Initialize a similarity score matrix (`similarity_scores`) to store the similarity measures. Rows represent documents and columns represents a word. Create a loop to loop through documents to calculate similarity score.

```
# Main Function: Calculate Similarity Scores between Documents and Queries
function calculate_similarity(csr_matrix, query_vector):
    Initialize similarity_scores array of size num_documents to 0

for each document i
        # Extract rows for document and query
        document_row = extract_row(csr_matrix, i)
        query_row = exstract_row(query_vector, 0)
```

2. Create a function that calculates the dot product of two CSR rows (shared word relevance).

```
# Helper Function: Calculate Dot Product of Two CSR Rows
function dot_product(row_a, row_b):
    Initialize dot_product to 0
    Initialize pointer_a to the start of row_a
    Initialize pointer_b to the start of row_b

    # Traverse both rows only where non-zero values are present
    while pointer_a < length(row_a) and pointer_b < length(row_b):
        if row_a.index[pointer_a] == row_b.index[pointer_b]:
            dot_product += row_a.value[pointer_a] * row_b.value[pointer_b]
            pointer_a += 1
            pointer_b += 1
        elif row_a.index[pointer_a] < row_b.index[pointer_b]:
            pointer_a += 1
        else:
            pointer_b += 1

    return dot_product
```

3. Create a function to calculate the magnitude (norm) of each vector, this calculates the Euclidean distance of a CSR row.

```
# Helper Function: Calculate Magnitude (Norm) of CSR Row
function calculate_norm(row):
    Initialize norm to 0
    for each value in row.values:
        norm += value^2
    return sqrt(norm)
```

4. Compute Cosine Similarity for each pair of documents i and j in the CSR matrix:
    a. Retrieve the CSR rows for documents `i` and `j`
    b. Compute Dot Product between rows i and j using `calculate_magnitude`

c. Calculate the Cosine Similarity using the formula `dot_product(row_i, row_j) / magnitude(row_i) x magnitude(row_j)`

d. Store the results in `similarity[i][j]` array

e. Return the `similarity_scores`

```
# Calculate dot product between document and query
numerator = dot_product(document_row, query_row)

# Calculate magnitudes (norms) of document and query
norm_doc = calculate_norm(document_row)
norm_query = calculate_norm(query_row)

# Compute cosine similarity, if denominator is non-zero
if norm_doc != 0 and norm_query != 0:
    similarity_scores[i] = numerator / (norm_doc * norm_query)
else:
    similarity_scores[i] = 0  # Assign 0 if either vector has zero magnitude
return similarity_scores
```

Question 2 (10 marks):

This question asks how the similarity sim(Q, D1) should change for different augmentations to D1, given:

- D1 = {Shipment of gold damage in a fire}
- Query = {gold silver truck}

The augmentations to consider are:

- (a) D1 = Shipment of gold damaged in a fire. Fire.
- (b) D1 = Shipment of gold damaged in a fire. Fire. Fire.
- (c) D1 = Shipment of gold damaged in a fire. Gold.
- (d) D1 = Shipment of gold damaged in a fire. Gold. Gold.

Note: No calculations are needed, just explain how the similarity should change.

Answer

Given:

- **D1 = {Shipment of gold damage in a fire}**
- **Query = {gold silver truck}**

(a) D1 = Shipment of gold damaged in a fire. Fire.

- *Effect:* Adding "Fire", which is **not in the query**, increases document length without increasing matching terms.
- **Similarity decreases** (due to increase in non-query term frequency).

(b) D1 = Shipment of gold damaged in a fire. Fire. Fire.

- *Effect:* Adding "Fire" again further increases non-query term frequency.
- **Similarity decreases further** (document vector grows even more with irrelevant terms).

(c) D1 = Shipment of gold damaged in a fire. Gold.

- *Effect:* Adding "Gold", which **is in the query**, increases the frequency of a query term.
- **Similarity increases** (matching term frequency increases).

(d) D1 = Shipment of gold damaged in a fire. Gold. Gold.

- *Effect:* Adding "Gold" again further increases frequency of a query term.
- **Similarity increases further** (more matching terms improve similarity).

Question 3 (10 marks):

This question is about term weighting schemes:

- Assuming the document collection consists of all scientific articles published in the Communications of the ACM (www.acm.org/dl)
- Identify two other sources of evidence (features or sets of features) that could be considered.
- Suggest a weighting scheme that incorporates these features.
- Define the evidence/feature, the reason for including it, and the means to include it in the weighting scheme.

Answer

To improve the term weighting scheme for scientific articles in the Communications of the ACM, I will use two additional sources of evidence that leverages semi-structured features.

- **Keywords as Contextual Indicators**
- **Citation Count as Importance Indicators**

Keywords as Contextual Indicators

Feature: Keywords assigned to each article, this could be the author or we could use keyword extraction methods to find meaningful insights.

Reasons for including: Keywords directly represent the main topics or themes of an article, offering clear indicators of the report. This feature will allow you to identify and highlight terms that are contextually important but may not appear as frequently in the main text.

Incorporation into Weighting Scheme: Assign a higher weight to terms identified as keywords to reflect their contextual importance. To do this you can multiply the term frequency of each keyword by a relevance factor in a TF-IDF weighting scheme. Example: the term ‘Gold’ appears in the query and has three appearances in a document D1, therefore the term frequency would be multiplied by 2 as a means of incorporating the weighting scheme. This boosts keywords relevance, making them more influential in retrieval.

Citation Count as Importance Indicators

Feature: The number of times an article has been cited by other articles.

Reasons for including: Citation count reflects the impact or relevance of an article within the field. Highly cited articles are often more influential, suggesting that terms within these documents are likely more relevant to the overall field.

Incorporation into Weighting Scheme: Use the citation count as a global weight factor. Calculate a citation weight for each article base on its relative citation frequency and apply this as a multiple to the TF-IDF weights of terms within that article. This makes terms in highly-cited articles more significant.

References

1. Nazli Goharian, Ankit Jain, Qian Sun, "Comparative Analysis of Sparse Matrix Algorithms For Information Retrieval", Available at: " https://ir.cs.georgetown.edu/downloads/SCI-Journal-CameraReady-Goharian.pdf".
2. Jain Anurag, 2024. 'TF-IDF Vectorization with Cosine Similarity', Medium, 4th Feb. Available at: 'https://medium.com/@anurag-jain/tf-idf-vectorization-with-cosine-similarity-eca3386d4423'
3. **Sparse Matrix Representations | Set 3 (CSR), 29 Nov, 2022.** Available at: https://www.geeksforgeeks.org/sparse-matrix-representations-set-3-csr/